جلسهٔ ۱۶ — یادگیریِ انتقالی و پروژهٔ نهاییِ DocScan (Transfer Learning)

جزوهٔ درس — هفتهٔ شانزدهم — پایانی

نگاهِ کلی به این جلسه

به جلسهٔ پایانیِ درس رسیدیم. مسیرِ ما یک قوسِ کامل داشت: در هفتهٔ ۱۲ دیدیم که ویژگی‌های دست‌طراحی (HOG) روی خودروهای شبیه‌به‌هم روی دقتِ **۴۸٪** متوقف شدند؛ در هفتهٔ ۱۳ فهمیدیم هاCNN ویژگی‌ها را خودشان یاد می‌گیرند اما به دادهٔ زیاد نیاز دارند. امروز گرهِ ماجرا باز می‌شود: با **یادگیریِ انتقالی** (transfer learning) از شبکه‌ای که قبلاً روی میلیون‌ها تصویر آموزش دیده استفاده می‌کنیم و — روی همان داده و با همان طبقه‌بند — دقت را از ۴۸٪ به **۸۷٪** می‌رسانیم. سپس کلِ خط‌لولهٔ **DocScan** را که هر هفته یک تکه‌اش را ساختیم، سرِهم می‌کنیم.

> **یادداشت**
>
> پیامِ محوریِ امروز: در بیشترِ پروژه‌های واقعی داده کم است. یادگیریِ انتقالی همان چیزی است که به شما اجازه می‌دهد با داده‌ی کم هم از قدرتِ یادگیریِ عمیق بهره ببرید. این، پرکاربردترین تکنیکِ عملیِ یادگیریِ عمیق است.

یک تشبیهِ ساده

تصور کنید می‌خواهید زبانِ ایتالیایی یاد بگیرید. دو نفر را در نظر بگیرید: یکی که هیچ زبانی جز فارسی نمی‌داند، و دیگری که پیش‌تر اسپانیایی آموخته است. نفرِ دوم بسیار سریع‌تر یاد می‌گیرد، چون بسیاری از ساختارها، ریشه‌ها و مفاهیم را از اسپانیایی به ایتالیایی منتقل می‌کند. لازم نیست او از صفر شروع کند. یادگیریِ انتقالی دقیقاً همین است: شبکه‌ای که «زبانِ دیدن» را روی میلیون‌ها تصویرِ ImageNet آموخته، بیشترِ آن دانش را به تسکِ جدیدِ شما (تشخیصِ خودرو یا خواندنِ کنتور) منتقل می‌کند و فقط باید «واژگانِ» تخصصیِ تسکِ شما را یاد بگیرد. به همین دلیل با دادهٔ کم و آموزشِ کوتاه به نتیجهٔ خوب می‌رسیم.

۱ مسئلهٔ دادهٔ کم

چرا نمی‌توانیم صرفاً یک CNN بزرگ روی داده‌های خودمان آموزش دهیم؟ چون هایCNN عمیق میلیون‌ها پارامتر دارند و برای برازشِ بدونِ بیش‌برازش به تا تصویرِ برچسب‌دار نیاز دارند. ما فقط ۶۰ تصویر در هر مدلِ خودرو داریم؛ آموزشِ یک شبکهٔ بزرگ از صفر روی این داده به‌شدت بیش‌برازش می‌کند و روی آزمون فرو می‌ریزد.

از سوی دیگر، سقفِ ۴۸٪ ویژگیِ دستی نشان داد که HOG هم کافی نیست. پس در یک بن‌بست به‌نظر می‌رسیم: نه دادهٔ

کافی برای یادگیریِ عمیق داریم، نه ویژگیِ دستیِ قدرتمند.

نکتهٔ کلیدی

راهِ خروج از این بن‌بست، مشاهده‌ای ظریف است: لازم نیست ویژگی‌ها را ما یاد بگیریم و لازم هم نیست از صفر یادشان بگیریم. می‌توانیم ویژگی‌هایی را که یک شبکهٔ دیگر روی دادهٔ عظیم یاد گرفته «قرض» بگیریم.

۲ بینشِ کلیدیِ یادگیریِ انتقالی

یک CNN که روی ImageNet (۱٫۲ میلیون تصویر، ۱۰۰۰ کلاس) آموزش دیده، ویژگی‌های بصریِ عمومی یاد گرفته است: لبه‌ها، بافت‌ها، شکل‌ها، اجزا. نکتهٔ حیاتی این است که این ویژگی‌های اولیه و میانی برای تقریباً هر تسکِ تصویری مفیدند — از جمله تشخیصِ مدلِ خودرو.

یادداشت

پس ایدهٔ یادگیریِ انتقالی ساده است: به‌جای یادگرفتنِ ویژگی‌ها از صفر، آن‌ها را از یک شبکهٔ ازپیش‌آموخته بازاستفاده کنیم. لایه‌های نخستِ یک شبکهٔ بینواییِ خوب، یک «استخراج‌گرِ ویژگیِ همه‌منظوره» است؛ هدردادنش عاقلانه نیست.

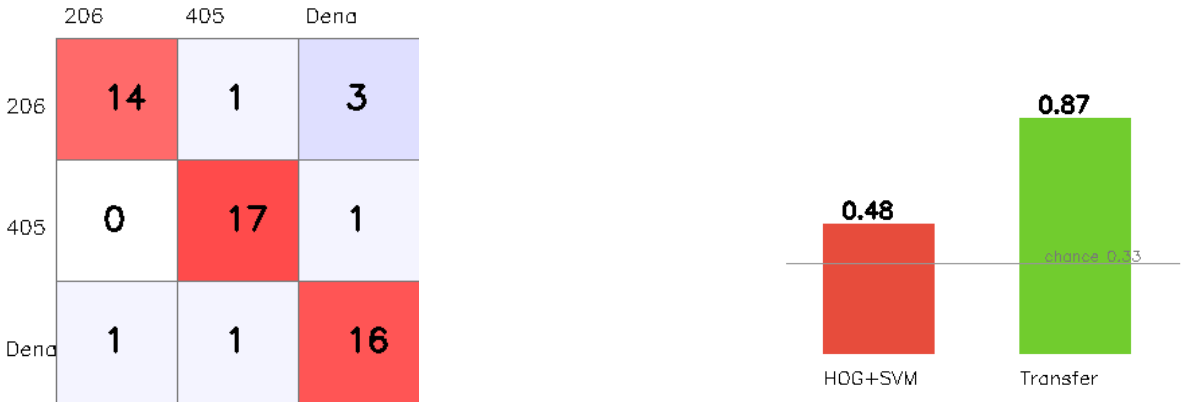
۱.۲ دو شیوهٔ یادگیریِ انتقالی

شیوه	چه می‌کنید	کِی
استخراجِ ویژگی	شبکه را منجمد کن، فقط یک سرِ طبقه‌بندِ نو آموزش بده	دادهٔ خیلی کم
تنظیمِ دقیق (fine-tune)	برخی لایه‌های بالا را باز کن و آموزش بده	دادهٔ بیشتر

ما از **استخراجِ ویژگی** استفاده کردیم: یک شبکهٔ ConvNeXt منجمدِ آموخته‌شده‌رویImageNet هر خودرو را به یک بردارِ ۱۰۲۴بُعدی تبدیل می‌کند، و سپس یک SVM طبقه‌بندی می‌کند — دقیقاً همان طبقه‌بندِ هفتهٔ ۱۲، فقط با ویژگی‌های بهتر.

۳ نتیجهٔ ما: سقف بالا می‌رود

نتیجهٔ واقعیِ اجرا روی همان مسئلهٔ ۴۰۵/۲۰۶/دنا:



چپ: مقایسهٔ دقت (HOG+SVM) ۴۸٫۰ در برابرِ انتقالی ۸۷٫۰ راست: ماتریسِ درهم‌ریختگیِ انتقالی

دقت کنید که چه چیزی تغییر نکرد: همان ۱۸۰ تصویر، همان SVM، همان تقسیمِ آموزش/آزمون. تنها تغییر: HOG جای خود را به ویژگی‌های منجمدِ ImageNet داد. نتیجه: **۴۸٫۰ به ۸۷٫۰**. یعنی ویژگی‌ها از همان ابتدا گلوگاه بودند، نه طبقه‌بند.

نکتهٔ کلیدی

ماتریسِ درهم‌ریختگیِ سمتِ راست حالا قطرِ غالب دارد (بیشترِ خودروها درست طبقه‌بندی شده‌اند)، در مقابلِ ماتریسِ تقریباً تصادفیِ هفتهٔ ۱۲. شبکهٔ ازپیش‌آموخته تفاوت‌های ظریفی را «می‌بیند» که HOG از دست می‌داد — مثلِ شکلِ جلوپنجره و الگوی چراغ‌ها.

۱.۳ چرا ۱۰۲۴ بُعد بهتر از ۹۰۰ بُعدِ HOG است؟

شاید بپرسید فرقِ بردارِ ۱۰۲۴بُعدیِ عمیق با بردارِ ۹۰۰بُعدیِ HOG در چیست — هر دو که فقط عدداند. تفاوت در معنای این اعداد است.

ریاضیات

هر عددِ HOG می‌گوید «در این سلول، چقدر لبه در این جهت هست» — یک ویژگیِ سطح‌پایین و ثابت. اما هر عددِ بردارِ عمیق، خروجیِ یک آشکارسازِ آموخته‌شده روی میلیون‌ها تصویر است و می‌تواند مفاهیمِ سطح‌بالا را کد کند: «چقدر شبیهِ جلوپنجرهٔ مشبک است»، «چقدر الگوی چراغِ کشیده دارد». این ویژگی‌های معنایی، کلاس‌ها را در فضای ۱۰۲۴بُعدی بسیار جداپذیرتر می‌کنند.

یادداشت

به عبارتِ دیگر، بُعدها مهم نیستند، کیفیتِ بُعدها مهم است. ۱۰۲۴ عددِ معنادار از یک شبکهٔ آموخته، بسیار ارزشمندتر از ۹۰۰ عددِ سطح‌پایینِ دستی است. همین، جهشِ ۴۸٫۰ به ۸۷٫۰ را توضیح می‌دهد.

۲.۳ چرا این‌قدر خوب کار می‌کند؟

- ImageNet شاملِ خودروهای فراوان است، پس شبکه از قبل اجزای مرتبط با خودرو را یاد گرفته.
- ویژگی‌های سطحِ‌میانیِ آن (شکلِ جلوپنجره، الگوی چراغ) دقیقاً همان چیزی است که مدل‌ها را از هم جدا می‌کند.
- قدرتِ یک شبکهٔ عظیم را با فقط ۶۰ تصویر در هر کلاس و چند ثانیه آموزش به دست می‌آوریم.

۳.۳ کدامَ لایه چه چیزی منتقل می‌کند؟

همهٔ لایه‌های یک شبکه به یک اندازه قابلِ انتقال نیستند:

- **لایه‌های اولیه:** لبه‌ها، رنگ‌ها، بافت‌ها — جهانی‌اند و به هر تسکی منتقل می‌شوند.
- **لایه‌های میانی:** اجزا و نقش‌مایه‌ها — به دامنه‌های مرتبط خوب منتقل می‌شوند.
- **لایه‌های پایانی:** مخصوصِ کلاس‌اند (مثلاً «سگِ نژادِ خاص») — کمترین قابلیتِ انتقال؛ اغلب جایگزین می‌شوند.

ریاضیات

قاعدهٔ سرانگشتی: هرچه تسکِ شما از ImageNet متفاوت‌تر باشد، باید لایه‌های پایانیِ بیشتری را دوباره آموزش دهید. برای عکس‌های معمولی (خودرو، کنتور) که «فاصلهٔ دامنه» (domain gap) کوچک است، انجمادِ کامل + استخراجِ ویژگی کافی است — و همین ۸۷٪ ما را داد.

۴.۳ تنظیمِ دقیق (fine-tuning) چطور کار می‌کند؟

اگر دادهٔ بیشتری داشتید و استخراجِ ویژگیِ ساده کافی نبود، گامِ بعدی تنظیمِ دقیق است:

۱. از وزن‌های ازپیش‌آموخته شروع کن (نه از وزن‌های تصادفی).

۲. لایهٔ طبقه‌بندیِ آخر را با یک لایهٔ نو برای تعدادِ کلاس‌های خودت جایگزین کن.

۳. لایه‌های اولیه را منجمد نگه دار و فقط لایه‌های بالایی را با نرخِ یادگیریِ کوچک آموزش بده.

هشدار / دام

نرخِ یادگیری در تنظیمِ دقیق باید کوچک باشد (مثلاً ۱۰ تا ۱۰۰ برابر کوچک‌تر از آموزش از صفر). وگرنه گام‌های بزرگ، دانشِ باارزشِ ازپیش‌آموخته را «فراموش» می‌کنند — پدیده‌ای به نامِ «فراموشیِ فاجعه‌بار» (catastrophic forgetting).

۵.۳ افزایشِ داده برای دادهٔ کم

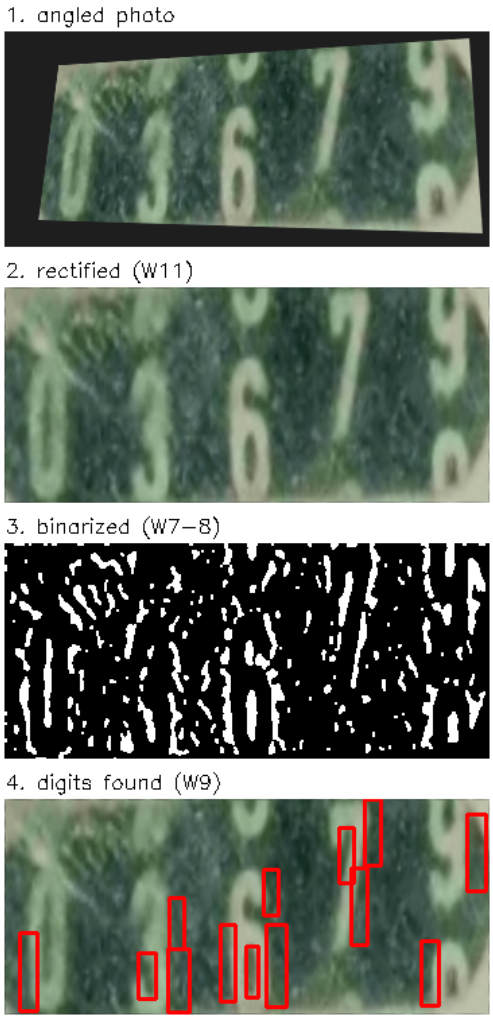
یک هم‌دستِ عالی برای یادگیریِ انتقالی، **افزایشِ داده** (data augmentation) است: داده را رایگان چندبرابر کنید با چرخش‌های کوچک، وارونه‌سازیِ افقی، تغییرِ روشنایی/کنتراست و برش‌های تصادفی. هر گونه، همان برچسب را زیرِ یک تغییرِ بی‌اهمیتِ جدید به شبکه می‌آموزد و در نتیجه ناوردایی بیشتر و بیش‌برازشِ کمتر می‌دهد.

نکتهٔ کلیدی

افزایشِ داده ارزان‌ترین دقتی است که می‌خرید؛ روی مسائلِ دادهٔ کم همیشه امتحانش کنید. اما مراقب باشید: افزایشی که برچسب را عوض کند بد است — مثلاً وارونه‌سازیِ عمودیِ یک رقم، ۶ را به ۹ تبدیل می‌کند!

۴ خط‌لولهٔ کاملِ DocScan

حالا به هدفِ نهاییِ درس می‌رسیم: سرهم‌کردنِ کلِ سامانهٔ DocScan که هر هفته یک مهرهٔ آن را ساختیم. شکلِ زیر خط‌لوله را روی یک نوارِ ارقامِ کنتورِ واقعی نشان می‌دهد:



پنج مرحله: عکسِ کج → صاف‌سازی → باینری‌سازی → جداسازیِ ارقام → خواندن

۱. **عکس:** کنتوری که از زاویه عکس گرفته شده (ورودیِ خام).

۲. **صاف‌سازی (هفتهٔ ۱۱):** هوموگرافی عکس را به نمای روبه‌رو تبدیل می‌کند.

۳. **باینری‌سازی (هفته‌های ۸/۷/۲):** آستانه + مورفولوژی تصویر را تمیز می‌کند.

۴. **جداسازی (هفتهٔ ۹):** کانتورها کادرِ هر رقم را پیدا می‌کنند.

۵. **خواندن (هفتهٔ ۱۳):** یک CNN هر رقم را طبقه‌بندی می‌کند → «۰۳۶۷۹».

۱.۴ هر هفته یک مرحله بود

هفته‌ها	نقش در DocScan
۱ تا ۴: پیکسل، هیستوگرام، فیلتر، حذفِ نویز	تمیزکردنِ عکسِ خام
۵ تا ۶: لبه، فوریه	یافتنِ ساختار، حذفِ نویزِ تناوبی
۷ تا ۹: مورفولوژی، بخش‌بندی، کانتور	جداکردنِ ارقام
۱۰ تا ۱۱: ویژگی، هوموگرافی	صاف‌سازی به اسکنِ تخت
۱۲ تا ۱۶: یادگیریِ ماشین، CNN، انتقالی	خواندنِ ارقام

یادداشت

DocScan یک پروژهٔ جدا نبود؛ ستونِ فقراتی بود که هر هفته یک مهره به آن اضافه کردیم. همین نشان می‌دهد که یک سامانهٔ واقعیِ بینایی، زنجیره‌ای از تکنیک‌هاست، نه یک تکنیکِ واحد.

۵ کدِ یادگیریِ انتقالی

```python
import torch, torch.nn as nn
from torchvision.models import convnext_base, ConvNeXt_Base_Weights
from sklearn.svm import SVC

weights = ConvNeXt_Base_Weights.IMAGENET1K_V1
model = convnext_base(weights=weights).eval()      # pretrained
feat = nn.Sequential(model.features, model.avgpool, nn.Flatten())
pre = weights.transforms()          # use the model's OWN preprocessing!

def embed(bgr):                     # image -> 1024-D deep feature
    rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
    t = pre(torch.tensor(rgb).permute(2,0,1)).unsqueeze(0)
    with torch.no_grad():
        return feat(t).numpy().ravel()

X = [embed(img) for img in images]  # deep features, not HOG
clf = SVC(kernel='rbf', C=10).fit(Xtr, ytr)   # same SVM as Week 12
```

هشدار / دام

مهم‌ترین خطِ این کد `pre = weights.transforms()` است. هر شبکهٔ ازپیش‌آموخته انتظارِ پیش‌پردازشِ خاصِ خود را دارد (اندازهٔ ورودی، میانگین/انحرافِ‌معیارِ نرمال‌سازی). اگر پیش‌پردازشِ اشتباه بدهید، ویژگی‌ها بی‌معنی می‌شوند و دقت فرو می‌ریزد. همیشه از پیش‌پردازشِ رسمیِ خودِ مدل استفاده کنید.

۶ اشتباه‌های رایج در یادگیریِ انتقالی

- **پیش‌پردازشِ ناهماهنگ:** همان‌طور که گفتیم، رایج‌ترین و مخرب‌ترین اشتباه.
- **فراموشیِ حالتِ eval:** اگر `model.eval()` را فراموش کنید، لایه‌های BatchNorm/Dropout در حالتِ آموزش می‌مانند و ویژگی‌ها ناپایدار می‌شوند.
- **نرمال‌نکردنِ ویژگی‌ها پیش از SVM:** ویژگی‌های عمیق مقیاس‌های متفاوت دارند؛ استانداردسازی اغلب کمک می‌کند.
- **تنظیمِ دقیق با نرخِ یادگیریِ بزرگ:** دانشِ ازپیش‌آموخته را نابود می‌کند.
- **نشتِ داده:** استخراجِ ویژگی روی کلِ داده پیش از تقسیم، سپس تقسیم — همان دامِ هفتهٔ ۱۲.

نکتهٔ کلیدی

جالب است که بیشترِ این اشتباه‌ها دربارهٔ مدل نیستند، بلکه دربارهٔ داده و پیش‌پردازش‌اند. این یک درسِ تکرارشوندهٔ کلِ بخشِ یادگیری است: کیفیتِ داده و درستیِ خط‌لوله، بیش از انتخابِ مدل اهمیت دارد.

۷ مهندسیِ یک خط‌لولهٔ مقاوم

تفاوتِ یک «دمو» و یک «محصول» در مدیریتِ شکست است. یک دمو روی یک عکسِ تمیز کار می‌کند؛ یک محصول باید در برابرِ تاری، انعکاسِ نور، کجی و کثیفی دوام بیاورد.

- **دروازه‌های اطمینان (confidence gates):** اگر تعدادِ درون‌داده‌های RANSAC کم بود، به‌جای حدس‌زدن، عکسِ جدید بخواه.
- **بررسیِ سلامت (sanity checks):** خوانشِ کنتور تعدادِ ثابتی رقم دارد؛ خوانشِ با تعدادِ اشتباه را رد کن.
- **مسیرهای جایگزین (fallbacks):** اگر مسیرِ ویژگی شکست خورد، مسیرِ کانتور را امتحان کن و برعکس.
- **ثبتِ لاگ (logging):** کیفیتِ هر مرحله (خطای بازتصویر، اطمینانِ ارقام) را ثبت کن تا بتوانی شکست‌ها را عیب‌یابی کنی.

هشدار / دام

همیشه «برای شکست طراحی کنید». پرسشِ درست این نیست «آیا روی این عکس کار می‌کند؟» بلکه «وقتی عکسِ بد بیاید، سامانه چه می‌کند؟». پاسخِ خوب، ردکردنِ مؤدبانه و درخواستِ ورودیِ بهتر است، نه تولیدِ خروجیِ غلطِ بی‌سروصدا.

۸ صداقت، اخلاق و محدودیت‌ها

- عددِ آزمون، مواردِ شکست، و داده‌ای که روی آن آموزش دیده‌اید را گزارش کنید — نه فقط بهترین دمو.
- یک رقمِ کنتورِ اشتباه یعنی یک قبضِ اشتباه؛ نرخِ خطایتان را بدانید و خوانش‌های کم‌اطمینان را دروازه‌بندی کنید.
- مراقبِ سوگیریِ داده باشید: اگر همهٔ کنتورهای آموزشی تمیز و روشن باشند، انتظارِ شکست روی کنتورهای کثیف را داشته باشید.

نکتهٔ کلیدی

«۹۷٪ دقیق» بدونِ این سه چیز بی‌معنی است: روی چه داده‌ای، با کدام سنجه، و چطور شکست می‌خورد. یک مهندسِ خوب همیشه این سه را کنارِ هر عدد گزارش می‌کند.

۹ قوسِ کاملِ درس

از یک تک‌پیکسل آغاز کردیم و به یک شبکهٔ عصبیِ ازپیش‌آموخته رسیدیم. دو نیمهٔ درس مکملِ هم‌اند:

- **پردازش تصویرِ کلاسیک (هفته‌های ۱ تا ۱۱):** دقیق، قابلِ‌تفسیر، بدونِ نیاز به آموزش — هنوز برای هندسه و پاک‌سازی ضروری است.
- **یادگیری (هفته‌های ۱۲ تا ۱۶):** قدرتمند برای شناسایی، به‌ویژه با یادگیریِ انتقالی.
- **یک سامانهٔ واقعی از هر دو استفاده می‌کند:** کلاسیک برای آماده‌سازی، آموخته برای شناسایی.

۱.۹ مرورِ کاملِ ۱۶ هفته

هفته	موضوع	مهارتِ کلیدی
۱	پیکسل‌ها و هیستوگرام	نمایشِ تصویر، عملیاتِ نقطه‌ای
۲	عملیاتِ نقطه‌ای پیشرفته	کشِ کنتراست، گاما، یکسان‌سازی
۳	کانولوشن و فیلترها	کرنل‌ها، بلور، تیزسازی
۴	نویز و بازسازی	میانه، دوطرفه، NLM
۵	تشخیصِ لبه	سوبل، لاپلاس، کَنی
۶	حوزهٔ فرکانس	فوریه، فیلترِ بالا/پایین‌گذر، ناچ
۷	مورفولوژی	فرسایش، اتساع، باز/بسته، تاپ‌هت
۸	بخش‌بندی	آبریزگاه، مؤلفه‌های همبند، Otsu
۹	کانتور و شکل	مساحت، کادر، توصیف‌گرهای شکل
۱۰	ویژگی‌ها	هریس، ORB، تطبیق
۱۱	هوموگرافی	صاف‌سازیِ پرسپکتیو، RANSAC
۱۲	ویژگی به یادگیری	بیش‌برازش، HOG+SVM
۱۳	شبکه‌های کانولوشنی	فیلترهای آموخته، آموزش
۱۴	تشخیصِ اشیا	کادر، IoU، NMS، YOLO، mAP
۱۵	دنباله و OCR	رمزگشایی، فاصلهٔ ویرایش، CTC، CRNN،
۱۶	یادگیریِ انتقالی	بازاستفاده، DocScan کامل

۲.۹ انتخابِ ابزارِ درست

اگر نیاز دارید به…	سراغِ … بروید
انطباق/صاف‌سازی/اندازه‌گیریِ هندسی	کلاسیک (هوموگرافی، کانتور)
حذفِ نویز، آستانه، مورفولوژی	فیلترهای کلاسیک
شناسایی با دادهٔ کم	یادگیریِ انتقالی
شناسایی با دادهٔ زیاد	آموزش/تنظیمِ دقیقِ CNN

یادداشت

نشانهٔ یک مهندس: انتخابِ ساده‌ترین ابزاری که مسئله را به‌طورِ قابلِ‌اعتماد حل می‌کند — نه پیچیده‌ترین یا جدیدترین. گاهی یک آستانهٔ Otsu بهتر از یک شبکهٔ عمیق است، و گاهی برعکس.

۳.۹ یک کلمه دربارهٔ منابعِ محاسباتی

نکتهٔ عملیِ مهم: در این جلسه شبکهٔ ConvNeXt را فقط برای استنتاج (استخراجِ ویژگی) به کار بردیم، نه آموزش. این تفاوتِ بزرگی در منابع دارد:

- **استخراجِ ویژگی:** فقط یک گذرِ روبه‌جلو برای هر تصویر؛ حتی روی CPU در چند ثانیه برای ۱۸۰ تصویر انجام می‌شود. سپس SVM روی بردارها فوری آموزش می‌بیند.
- **آموزشِ کامل از صفر:** نیازمندِ GPU، ساعت‌ها یا روزها زمان، و دادهٔ عظیم.

نکتهٔ کلیدی

به همین دلیل یادگیریِ انتقالی به‌سبکِ استخراجِ ویژگی، «دموکراتیک‌ترین» شکلِ یادگیریِ عمیق است: بدونِ GPU گران و بدونِ دادهٔ عظیم، به دقتِ بالا می‌رسید. برای بسیاری از پروژه‌های صنعتیِ کوچک‌مقیاس (مثلِ خواندنِ کنتور)، همین رویکرد کافی و بهینه است.

۱۰ مسیرِ ادامه

- **تشخیصِ شیء** (YOLO، Faster-RCNN): یافتنِ و مکان‌یابیِ اشیا — گامِ بعد از طبقه‌بندی.
- **بخش‌بندیِ معنایی** (U-Net، SAM): برچسب‌زدنِ هر پیکسل — برای مجموعه‌داده‌های خطِ‌برق/ماسکِ شما.
- **سامانه‌های OCR** (Tesseract، TrOCR): خط‌لوله‌های کاملِ خواندنِ متن که روی همین ایده‌ها ساخته شده‌اند.
- **استقرار (deployment):** روی‌دستگاه (TFLite، ONNX)، تأخیر، و مقاومت در میدانِ واقعی.

۱۱ تمرینِ کلاسی و پاسخ

تمرین

۱) شما ۴۰ تصویر در هر کلاس دارید. CNN از صفر یا یادگیریِ انتقالی؟ چرا؟

۲) یادگیریِ انتقالی خودروها را از ۴۸٪ به ۸۷٪ رساند با همان SVM. دقیقاً چه چیزی عوض شد؟

۳) در DocScan، مرحلهٔ صاف‌سازی درون‌داده‌های بسیار کمی برمی‌گرداند. سامانه باید چه کند؟

پاسخ‌ها: (۱) **یادگیریِ انتقالی** — ۴۰ تصویر برای آموزشِ شبکهٔ عمیق از صفر بدونِ بیش‌برازش بسیار کم است. (۲) فقط **ویژگی‌ها**: از HOG دست‌طراحی به ویژگی‌های عمیقِ ازپیش‌آموخته؛ طبقه‌بند یکسان بود. (۳) **دروازه‌بندی کن**: صاف‌سازی را نامطمئن تلقی کن و به‌جای خواندنِ اسکنِ بد، عکسِ جدید بخواه.

۱۲ جمع‌بندیِ درس

در این جلسه یاد گرفتیم که یادگیریِ انتقالی چطور با بازاستفاده از یک شبکهٔ ازپیش‌آموخته، مشکلِ دادهٔ کم را حل می‌کند و — روی همان داده و طبقه‌بندِ هفتهٔ ۱۲ — دقت را از ۴۸٪ به ۸۷٪ رساند؛ سپس کلِ خط‌لولهٔ DocScan را سرِهم کردیم و دیدیم هر هفته یک مرحله از آن بوده؛ و آموختیم که مهندسیِ مقاوم یعنی دروازه‌های اطمینان، بررسیِ سلامت و مدیریتِ صادقانهٔ شکست. قوسِ کاملِ درس از یک پیکسل تا یک اسکنرِ کارآمد کشیده شد: پردازشِ کلاسیک برای آماده‌سازی، یادگیریِ عمیق برای شناسایی، و قضاوتِ مهندسی برای دانستنِ اینکه کدام، کِی. این پایانِ درس است — و آغازِ کارِ شما.